FUNDAMENTOS DE LA INTELIGENCIA ARTIFICIAL

CURSO 2025-2026

BUSCANDO AL CORONEL KURTZ

Autor:

Maite Viejo Baena

Madrid

Enero, 2026

Índice de Contenidos

Introducción 3
Parte 1 4
Entorno 4
Aproximación del problema 5
Inicialización del programa 6
Proceso de deducción del agente 7
Guiar manualmente 9
Parte 2 10
Parte 2.1 10
Entorno 10
Representación del conocimiento/ inferencia bayesiana 11
Representación de mapas de probabilidad o heatmaps 12
Toma de decisiones 13
Búsqueda lógica: 13
Búsqueda usuario: 13
Búsqueda 14
Lanzar granada 14
Parte 2.2 15
Entorno 15
Estados 15
Acciones 16
Recompensas 16
Transiciones 16
Value Iteration 17
Ejecución 18
Representación del entorno 18
Análisis 18

Introducción

Este documento expone el trabajo de Maite Viejo Baena, alumna de la Escuela Técnica Superior de Ingeniería (ICAI) de la Universidad Pontificia de Comillas. Dicho trabajo tiene como título “Buscando al Coronel Kurtz” y ha sido realizado en la asignatura de Fundamentos de la Inteligencia Artificial de 2º de iMAT.

Este proyecto consta de dos partes, cuyos módulos son:

- Parte 1:
  - kurtz.py
  - agente.py
  - busquedas.py
  - entorno.py
- Parte 2:
  - palacio.py
  - river_mdp.py
  - parte2.py
  - parte2_2.py
- requirements.txt

Parte 1

En esta parte de la memoria se expone el trabajo realizado en la primera parte del proyecto, donde, en su misión, el Capitán Willard ha de encontrar al coronel Kurtz dentro de un palacio lleno de trampas, tres precipicios y un soldado enemigo.

El objetivo de este proyecto es diseñar un agente capaz de explorar un entorno parcialmente observable utilizando razonamiento lógico con el fin de evitar peligros, además de localizar al coronel Kurtz y salir del entorno de manera segura.

Entorno

El entorno utilizado para este proyecto es el pedido en el enunciado, un tablero de 6 x 6 habitaciones, cuyos elementos vienen constituidos por tres precipicios, un soldado, la salida y el coronel.

Los elementos que constituyen el palacio pueden ser los siguientes:

- Brisa: percepto producido en las habitaciones contiguas de los precipicios
- Ronquido: percepto producido en las habitaciones contiguas del soldado
- Resplandor: percepto producido por las casillas contiguas de salida, además de la casilla de salida
- Paredes: percepto de las habitaciones que muestran si una habitación tiene una pared y por lo tanto no hay una nueva habitación es ese sentido

Para este proyecto, se denomina celda adyacente a una celda, aquellas que se encuentran a distancia Manhattan 1 de la celda que contiene el elemento.

Aproximación del problema

Para la resolución del problema se han implementado objetos, creando cuatro clases distintas:

- Palacio:
  Tablero de juego, que puede ser formado por dos tipos de habitaciones, formando un palacio de creencias que será el que utilizará el agente o un palacio que contenga los verdaderos valores de las casillas que sea el tablero del juego con todos los perceptos y elementos en el que irá 'descubriendo' el agente.

- Rooms:
  Objeto que define las propiedades de las habitaciones del palacio, es decir, los perceptos que contiene, si tiene paredes, el elemento que tiene la casilla, es decir, si se encuentra la salida o el soldado entre otros

- Creencias_Room:
  Objeto que define las habitaciones con los conocimientos que tiene el agente de cada habitación, si cree que puede ser esa casilla un precipicio soldado o donde puede que esté la salida.

- Agente:
  Clase que contiene todas sus acciones y con distintos atributos como el tablero, la ubicación en la que se encuentra o valores como las casillas que ya han sido visitados y las fronteras.
  Su tablero está constituido por celdas del tipo Creencias_Room.

El agente no tiene acceso directo al estado real del entorno, lo que realiza el agente es tener una representación interna que se basa en los perceptos que recibe.

Inicialización del programa

Este programa tiene dos modos distintos de operación.

- Guiar al agente manualmente, decidiendo el usuario que acción tomar, siendo aconsejado por los perceptos que recibe el agente (Capitán Willard) del entorno. Figura1

- Proceso de búsqueda: en este caso el agente busca por si mismo la ruta para lograr su objetivo de encontrar al coronel y salir del palacio. Figura2



FIGURA1

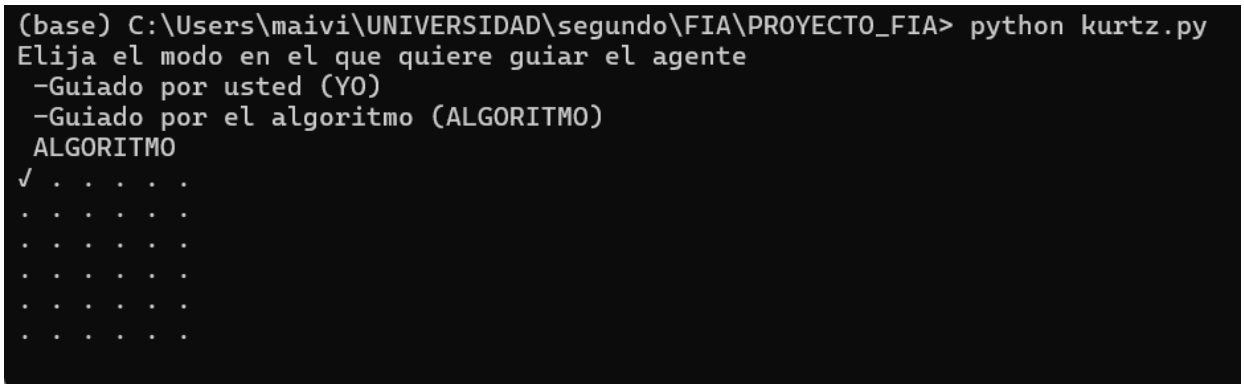


FIGURA 2

Proceso de deducción del agente

Mientras no se haya conseguido llegar a la salida cuando el Capitán Willard y el coronel Kurtz.

El agente, comienza por comprobar que la casilla en la que se encuentra no sea un obstáculo. A continuación, se llama a la función actualizar_casillas.

Actualizar_casillas:

Función que actualiza las creencias del agente en base a los conocimientos de la casilla actual. Lo primero que hace esta función es marcar la habitación como segura, además de visitada, añadiéndola al atributo de casillas visitadas y quitándola del atributo de las casillas frontera.

A continuación, se guardan sus casillas vecinas, pues, en el caso de haber brisa en la casilla actual, se puede deducir que alguna de sus casillas contiguas sea un precipicio, o cuando se perciben ronquidos, se puede deducir que en una de las casillas contiguas hay un soldado o si hay un resplandor, nos podemos encontrar en una casilla contigua o en la casilla de salida.

A continuación, se llama a las funciones de deducción_precipicio y deducción_soldado con la intención de obtener donde se encuentran estos obstáculos

Deduccion_precipicio:

Esta función, deduce la posición de los precipicios en función del conocimiento que tiene sobre el entorno, combinando todas las casillas visitadas que contienen brisa.

Se recorren todas las casillas que ya han sido visitadas, y de estas, se consideran sus casillas vecinas que no han sido visitadas, y, en caso de que en ellas haya un posible precipicio, se añade a la lista.

En caso de que hay más de dos conjuntos, para poder hacer la intersección de las casillas visitadas que contienen brisa, en caso de que la intersección sea un único elemento, significa que se ha encontrado un precipicio, y, por lo tanto, se puede guardar la casilla como tal.

Deduccion_soldado:

Por otro lado, para deducir un soldado, se deben de estudiar paso a paso las casillas vecinsa de las celdas visitadas. En caso de que una de las casillas visitadas tenga ronquidos, se exploran sus vecinos, y, si únicamente uno de sus vecinos tiene la posibilidad de ser un soldado, se guarda como que definitivamente lo es. Esto ocurre gracias a la actualización de creencias de la función anterior.

Además, debido a que hay un único soldado, una vez deducido cual es, se sabe que no hay ninguno más, y, por lo tanto, se puede eliminar esa posibilidad.

A continuación, en el algoritmo del agente, se actualiza la frontera de este. Esta actualización es sencilla, pues únicamente se recorren todos los vecinos de las habitaciones exploradas. En caso de que el vecino no haya sido explorado, se añade a la frontera.

En caso de que ya se sepa la posición del soldado y se encuentra en una casilla contigua, el agente lanza la granada al soldado con el fin de acabar con él.

Por otro lado, se comprueba que el agente no haya conseguido el estado objetico final (estar en la salida con el coronel), finalizando la partida.

Así mismo, en caso de que se encuentre con el coronel, pero no esté en la casilla de salida, hay dos opciones. La primera, que no se sepa la ubicación de la salida, y por lo tanto se continúe el algoritmo de **BFS** que se implementa en toda la búsqueda, o, por segunda alternativa, que se sepa la ubicación de la casilla de salida, y, por lo tanto, se busque, también por **BFS** usando colas el camino para llegar a la salida haciendo uso del atributo de casillas visitadas.

Por el contrario, si aún no se ha conseguido el objetivo, el algoritmo va creando unos objetivos que se podrían categorizar como instantáneos que son la siguiente casilla hacia la que se van a mover. Este objetivo 'temporal' se obtiene buscando la casilla más cercana a la actual a través del atributo de frontera, y a continuación, mediante el algoritmo de **BFS** se calcula el trayecto que ha de seguir el agente para llegar a él.

Busqueda_anchura:

Se comienza por inicializar una cola donde se van a ir añadiendo los vecinos seguros que no hayan sido visitados, además, se crea un conjunto que guarda las habitaciones visitadas y un diccionario para construir la ruta.

Mientras la cola no esté vacía, es decir, sea posible la expansión, se saca el elemento que lleve más tiempo en la cola, habiendo dos opciones. Que este elemento sea el que se buscaba como 'objetivo temporal' o que no. En caso de que lo sea, se va reconstruyendo el camino, por el contrario, se van añadiendo los vecinos seguros que no hayan sido visitados como comentado anteriormente.

En caso de que no se haya encontrado una ruta, significa que no se puede avanzar de forma segura y, por lo tanto, el agente finaliza.



FIGURA 3

Guiar manualmente

Este es el otro modo de operación del programa, donde el agente ha de razonar con las percepciones que tiene del entorno, pero la toma de decisión de la ruta ha de ser tomada por el usuario.

El modo de operación es parecido al del agente lógico. Se comienza por actualizar las casillas, deducir si hay precipicio o no en alguna casilla y de forma análoga buscar si hay un soldado.

El objetivo en este caso es el mismo que el anterior, salir del palacio con el coronel Kurtz. Sin embargo, el usuario participa en la toma de decisiones del agente.

En caso de que se encuentre el soldado, el usuario es el responsable de decidir si se lanza la granada o no. Además de que se le muestran todas las casillas que componen la frontera para este tomar la decisión de hacia dónde dirigirse.

Para que el usuario pueda tomar la decisión de manera correcta, dispone en todos los movimiento de un mapa del palacio donde cada situación es representada de distinta manera, aportando una mejor visión de las decisiones que se pueden tomar, como se muestra en la figura.

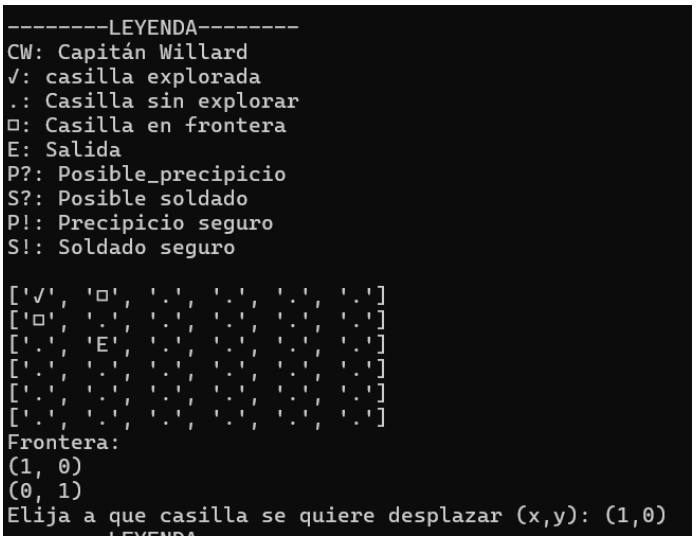


FIGURA 4

Parte 2

Esta segunda parte del proyecto se divide en dos partes distintas, por lo tanto, esta parte de la memoria va a también ser dividida en dos partes para explicar el razonamiento y maneras de atacar los distintos problemas

Parte 2.1

El objetivo de esta parte del proyecto es diseñar un agente inteligente capaz de tomar decisiones en un entorno (palacio) parcialmente observable. Este entorno esta dividido en distintas casillas que pueden tener distintos obstáculos (fuego, dardos, pinchos o soldado enemigo).

El Capitán Willard (agente) ha de moverse por el palacio con el objetivo final de encontrar al coronel Kurtz y encontrar la salida para salir del palacio.

Dado que el agente no tiene un conocimiento completo del entorno se hace uso de un enfoque bayesiano donde las creencias de localización de los siguientes obstáculos se representan mediante distribuciones de probabilidad que se van actualizando de forma dinámica a lo largo de que avance la partida.

Entorno

El entorno es modelado con un tablero de las dimensiones proporcionadas en las instrucciones del proyecto (6x6), donde cada celda representa una habitación del palacio. Cada una de estas casillas pueden contener:

- Trampas mortales:
    - Fuego
    - Pinchos
    - Dardos
    - Soldado enemigo dormido: este soldado se puede matar haciendo uso de la granada que tiene el Capitán Willard, la única condición es que hay una única oportunidad y debe de ser lanzada desde una casilla contigua a donde ubica.
- Salida
- Coronel Kurtz
- Estar vacía

Además, cada una de las casillas contiguas a una de las trampas mortales o de la salida tiene unos perceptos que advierten al Capitán de su presencia y son los usados para obtener su conocimiento. Además, también contiene la existencia de las paredes que delimitan el movimiento del agente.

Se entiende por casilla contigua, de forma análoga a la parte 1 de la práctica a aquellas casillas que se encuentren a distancia Manhattan 1 de la casilla que contiene el peligro.

Representación del conocimiento/ inferencia bayesiana

El conocimiento del agente es representado por distintos mapas de probabilidad que son independientes entre sí y se van actualizando en cada movimiento, para cada elemento relevante del entorno contiene un mapa propio:

- Mapa de probabilidad de fuego
- Mapa de probabilidades de pinchos
- Mapa de probabilidad de dardos
- Mapa de probabilidad de la ubicación del soldado enemigo
- Mapa de probabilidad de salida

A medida que va avanzando el agente por el tablero, este va actualizando sus creencias aplicando el teorema de Bayes, de tal manera que para cada percepto observado en una casilla, se ajustan las probabilidades de las casillas vecinas y de la misma casilla según la evidencia encontrada en cada casilla.

Esta probabilidad comienza en todas las casillas de todos los mapas con un prior uniforme, para reflejar su falta de conocimiento sobre la posición de cualquiera de los elementos del palacio:

$$P(\tau_{(i,j)}) = \frac{1}{N-1}$$

A medida que se avanza en la ejecución del programa, se tiene la función de verosimilitud tal que teniendo en cuenta la información que conocemos acerca de los estímulos:

$$P(e^{\tau}_{(i.j)}|\tau_{(k,l)}) = \begin{cases} 1 & si\ (k,l) \in adj(i,j) \cup \{(i,j)\} \\ 0 & si \quad cualquier\ otro\ caso \end{cases}$$

De esta forma, calculamos un posterior que cumple la fórmula:

$$P(\tau_{(k,l)} \mid e^{\tau}_{(i.j)}) = \frac{P(e^{\tau}_{(i.j)}|\tau_{(k,l)}) * P(\tau_{(k,l)})}{P(e_{\tau})}$$

Siendo luego calculada la constante de normalización para que la suma de todas las celdas por tablero sea 1.

Como síntesis de la forma en la que ha sido programado el script, cuando un estímulo no es percibido, la hipótesis generada es que ni la casilla actual ni sus contiguas contienen el peligro vinculado al peligro y por lo tanto, la probabilidad de esa casilla en el mapa correspondiente será 0, pues según la función de verosimilitud, la probabilidad de esta es 0, y por lo tanto, su posterior también lo será.

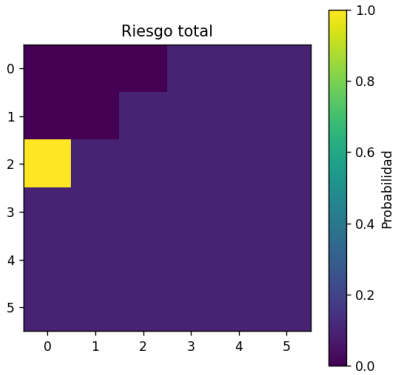
Este proceso por medio de la inferencia bayesiana permite que el agente vaya progresivamente actualizando su conocimiento para que consiga su objetivo final.

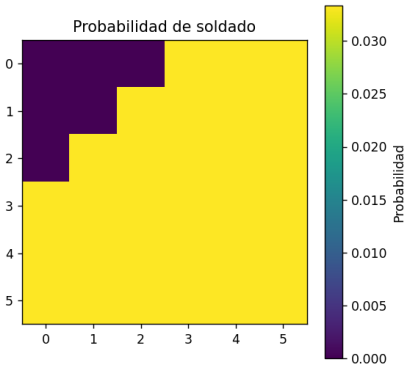
Representación de mapas de probabilidad o heatmaps

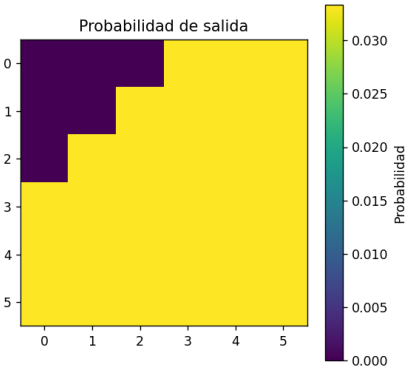
A partir de los mapas de las probabilidades anteriores explicados anteriormente, se construye un mapa de probabilidad de riesgo total. Para el cálculo de este mapa se ha seguido la opción propuesta en el enunciado de este proyecto que consiste en que cada casilla de este mapa está constituida por la suma de todas las probabilidades de los riesgos en la casilla correspondiente. Aunque la suma de todas las casillas no sea uno, este mapa proporciona una idea aproximada del riesgo que presenta cada una de las casillas teniendo en cuenta todos los peligros.

Para facilitar la toma de decisiones del usuario cuando tiene que tomar la decisión de hacia que coordenada de la frontera, como lo pedido en el enunciado, se proporcionan tres heatmaps distintos. Estos heatmaps corresponden a:

- Heatmap de las probabilidades de la posición de la casilla de salida
- Heatmap de las probabilidades de la posición del soldado enemigo
- Heatmap de las probabilidades de la posición del riesgo total.







Como se puede observar en los gráficos mostrados, los colores más azules, significa que hay una menor probabilidad de que haya en esa casilla, mientras que los colores amarillos son los que tienen más probabilidad de todo el gráfico.

En este caso, auqnue se mantengan todos los mapas individuales internos, se ha decidido mostrar únicamente estos tres pues son los más relevantes para la toma de decisiones.

Toma de decisiones

Búsqueda lógica:

La forma en la que en este caso el agente toma decisiones es haciendo uso de la estrategia greedy, diferenciando en dos casos:

- Si ***NO*** se ha encontrado al coronel Kurtz:

  En este caso, el objetivo en cada ronda del agente es buscar la siguiente casilla que parezca más segura a la que se pueda acceder. La heurística que se calcula para la toma de decisiones en este algoritmo es el riesgo total de las casillas, expandiéndose progresivamente con la intención de aumentar su conocimiento.

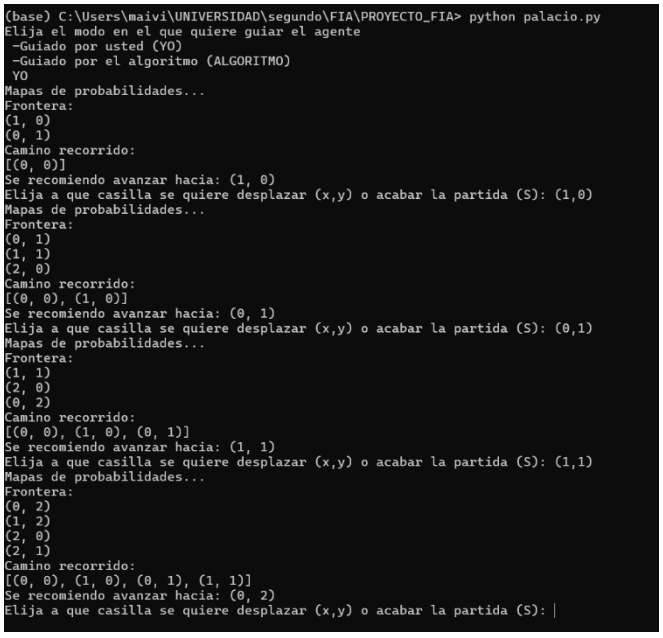
- Si ***SÍ*** se ha encontrado al coronel Kurtz:

  En este otro caso, el objetivo de cada ronda es ir acercándose progresivamente a la salida, por lo tanto, el objetivo temporal del agente es moverse a la casilla que tenga una mayor probabilidad de ser la casilla de salida, esto se hace mediante una función auxiliar que explora el mapa de probabilidad de la salida. La heurística en este caso, para decidir que casilla será el siguiente paso, es la distancia Manhattan de la casilla hacia la casilla objetivo del momento.

El cambio de la heurística y la aproximación a la toma de decisiones mediante el algoritmo greedy, se realiza con la intención de evitar la exploración innecesaria del tablero una vez cumplida la mitad del objetivo final (haber encontrado al coronel), exponiendo al agente a un menor riesgo a que si estuviera explorando por el resto del tablero.

Búsqueda usuario:

En este otro caso, es el usuario el que toma la decisión de hacia que casilla de la frontera del agente avanzar, sin embargo, en este caso, además de proporcionar al usuario con los heatmaps de riesgos total, ubicación del soldado y ubicación de la salida, se le proporciona una sugerencia que está basada en el pensamiento greedy del agente lógico, es decir, esta sugerencia es la acción que tomaría el agente si tuviera que tomar la decisión por lógica como en el caso anterior. Además, también se le proporciona una lista con el recorrido que ha realizado, de tal manera que puede retroceder sobre sus pasos si lo desea, o, salir de la partida si quiere dejar de jugar.



Búsqueda

El agente hace uso de una frontera compuesta por las casillas vecinas de las habitaciones ya visitadas, de tal forma que sobre estas casillas que componen la frontera se aplica el algoritmo greedy.

Sin embargo, el algoritmo puede presentar situaciones en las que todas las opciones de la frontera superan el umbral de riesgo impuesto en el programa (0.2) y, por lo tanto, se ha implementado un mecanismo de backtracking, donde el agente regresa a casillas previamente vistas con la intención de explorar nuevas ramas, evitando bloqueos locales y bucles infinitos.

Esto garantiza una exploración del entorno robusta y completa.

Lanzar granada

Para que esta acción se ejecute de manera correcta, el agente ha de encontrarse en una celda contigua a la del soldado para poder lanzarla y que caiga sobre este acabando con él, por ello en el caso del agente lógico, antes de realizar la acción, se asegura de que sí que cumple con las condiciones para lanzarla de manera correcta.

Por otro lado, en la búsqueda del usuario, se le dice una vez encontrado el soldado si desea lanzar la bomba, en caso de que diga que no, se continua con el proceso, si dice que si, se lanza la bomba hacia la casilla que indique, con la intención de que se la lance al soldado.

Parte 2.2

En esta segunda parte el problema que se aborda es distinto al de las otras partes. En esta parte el capitán Willard debe desplazarse junto al coronel Kurtz de una orilla del río a otra, llegando a la salida, enfrentándose a un entorno estocástico con una fuerza de corriente. En este caso, el entorno es completamente observable, pero las consecuencias de las acciones no son deterministas.

El principal objetivo es modelar el problema como un proceso de decisión de Markov, resolviéndolo mediante Value Iteration, obteniendo la política óptima que maximice la recompensa esperada acumulada. Esta política será posteriormente ejecutada en el entorno real, simulando el comportamiento del agente bajo incertidumbre.

Entorno

El entorno ha sido modelado con una lista de listas de 7 filas y 6 columnas, donde se pueden encontrar:

- Columna 0: vacía excepto la posición de llegada del coronel con el Capitán
- 2 islas peligrosas en las cuales, si pisan, mueren
- Columna 6: vacía a excepción de la casilla de salida donde se acaba la partida

El movimiento del agente por el río se ve afectado por la corriente del río, cuya intensidad depende en cada columna por la generada en la función river_strenght().

Estados

Los estados son representados por las coordenadas actuales en las que se encuentra el agente en el tablero, con el par (x,y), siendo todos ellos considerados en el proceso de planificación.

Los estados terminales en este caso son la salida y las casillas que contiene una isla. Una vez determinado un estado terminal, el proceso se detiene y no se realizan más transiciones. Durante la planificación, estos estados se incluyen explícitamente para asegurar la correcta propagación de los valores en la iteración.

Acciones

Las acciones que se pueden realizar son:

- Arriba
- Abajo
- Izquierda
- Derecha

Siendo la opción de mantenerse en una misma casilla como resultado de una transición cuando el movimiento no es posible por un borde o isla, no se toma como una acción en sí, sino como consecuencia.

Recompensas

Las funciones de recompensas vienen definidas en el enunciado como:

- +100: alcanza la salida
- -100: alcanza una isla
- -1: por cada movimiento realizado que no sea caer en una isla o salida.

De esta manera, se penalizan las trayectorias largas y se fomenta que el agente llegue a una solución

Transiciones

Las acciones no siempre tienen un resultado determinista condicionado por las corrientes del río y la presencia de obstáculos. Cada una de las acciones (a) que se pueden tomar en un estado (s) está asociada a una distribución de probabilidad tal que:

$$\mathrm{P}(\mathrm{s}'\,|\mathrm{s},\mathrm{a}) = \begin{cases} P_{dir} = 1 - river_strenght(j), & s' \; casilla \; en \; dirección \; a \; con \; a \neq down \\ P_{dir} = 1, & si \; a = down \; y \; s' casilla \; inferior \\ P_{down} = river_strenght(j), & si \; s' es \; la \; casilla \; inferior \; y \; a \neq doown \\ P_{stay} = 1, & si \; s' = s \; por \; un \; borde \; o \; isla \\ 0, & en \; cualquier \; otro \; caso \end{cases}$$

Value Iteration

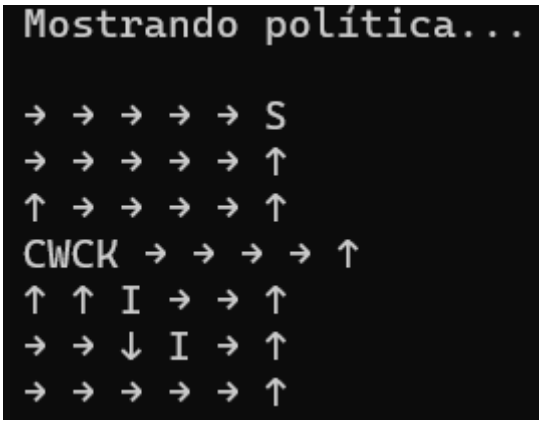
Para resolver el proceso de decisión de Markov (MDP) se emplea, según lo pedido, el algoritmo de Value Iteration, cuya definición según lo explicado en caso es:

$$V(s) = \max_a \sum_{s'} P(s'|s,a)[R(s,a,s') + \gamma V(s')]$$

En este caso utilizamos un valor $\gamma = 0.9$, que permite buscar un equilibrio entre las recompensas a fututo y las obtenidas en el momento. El proceso iterativo se detiene cuando el cambio máximo entre iteración e iteración es menor que el umbral ε, es decir, cuando las iteraciones convergen hacia un valor.

Como resultado de aplicar este algoritmo se obtienen dos cosas:

- La función del valor V(s) que corresponde a la recompensa esperada en cada estado
- La política óptima $\pi(s)$ que indica cual es la acción óptima para realizar en cada estado



Ejecución

A diferencia de en la planificación, durante la ejecución el algoritmo no vuelve a planificar su ruta ni evalúa su calidad, sino que se utiliza la política generada en la planificación.

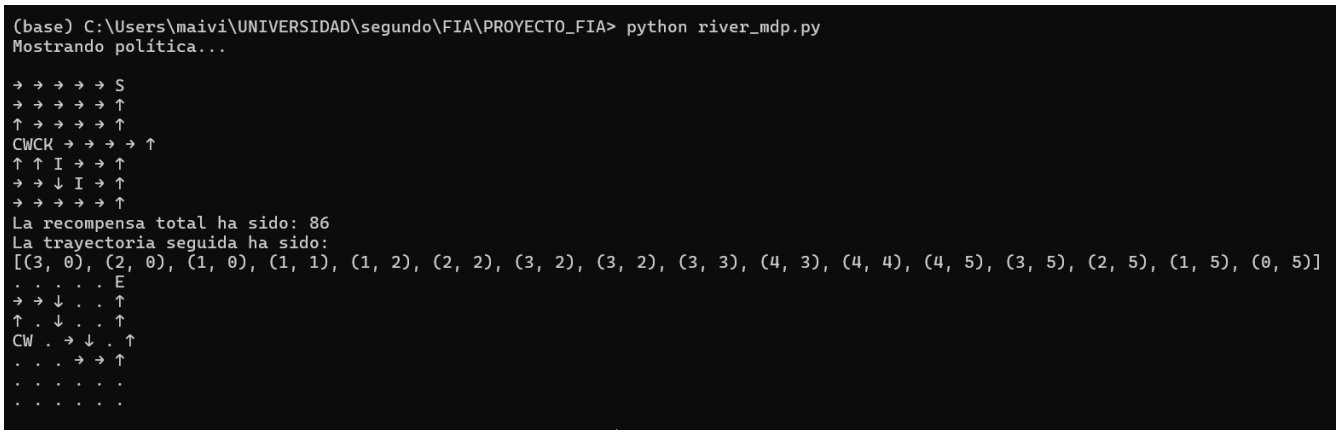
Es decir, en la ejecución, el agente no recalcula la política, sino que se limita a seguir la política calculada, estando sujeto a la incertidumbre del entorno a través de las transiciones estocásticas.

De este modo, aunque la política haya sido calculada bajo un modelo conocido, la trayectoria seguida durante la ejecución puede diferir entre simulaciones, reflejando la incertidumbre del entorno.

Representación del entorno

Para facilitar la interpretación de los resultados, he realizado dos gráficas distintas. La primera de ellas representa la política óptima calculada mediante Value Iteration, mientras que la segunda representa el trayecto tomado por el agente.

Las flechas muestran el movimiento que realiza el agente en dicha casilla y como se puede observar en ambos casos, se mantienen marcadas tanto la casilla de entrada como de salida.



Análisis

Los resultados nos muestran que el agente es capaz de llegar a la salida maximizando la recompensa incluso si se presentan corrientes que introducen desviaciones aleatorias en la trayectoria.

Además, cabe recalcar que hay casos en los que la política óptima no coincide con el trayecto del agente debido al carácter estocástico del entorno.

Partidas extra

```
Mostrando política...

→ → → → → ↓
→ → → ↓ I ↓
→ → → ↓ I ↓
CWCK → → → → S
→ → → → → ↑
→ → → → → ↑
→ → → → → ↑
La recompensa total ha sido: 78
La trayectoria seguida ha sido:
[(3, 0), (3, 1), (4, 1), (5, 1), (6, 1), (6, 2), (6, 2), (6, 2), (6, 2), (6, 3), (6, 3), (6, 3), (6, 3), (6, 3), (6, 3), (6, 3), (6, 3), (6, 3), (6, 3), (6,
 4), (6, 5), (5, 5), (4, 5), (3, 5)]
. . . . . .
. . . . . .
. . . . . .
CW ↓ . . . E
. ↓ . . . ↑
. ↓ . . . ↑
. → → → → ↑
```

```
Mostrando política...

→ → → → → ↓
→ → → → → ↓
→ → → → → ↓
CWCK I → ↑ I S
→ → → → → ↑
→ → → → → ↑
→ → → → → ↑
La recompensa total ha sido: 88
La trayectoria seguida ha sido:
[(3, 0), (2, 0), (2, 1), (2, 2), (2, 3), (3, 3), (4, 3), (5, 3), (6, 3), (6, 4), (6, 5), (5, 5), (4, 5), (3, 5)]
. . . . . .
. . . . . .
→ → → ↓ . .
CW . . ↓ . E
. . . ↓ . ↑
. . . ↓ . ↑
. . . → → ↑
```

```
Mostrando política...

→ → → → → ↓
→ → → → → S
↑ → ↑ I → ↑
CWCK → → → → ↑
→ → → → → ↑
↑ I → → → ↑
→ → → → → ↑
La recompensa total ha sido: 90
La trayectoria seguida ha sido:
[(3, 0), (3, 1), (4, 1), (4, 2), (5, 2), (5, 3), (5, 4), (5, 5), (4, 5), (3, 5), (2, 5), (1, 5)]
. . . . . .
. . . . . E
. . . . . ↑
CW ↓ . . . ↑
. → ↓ . . ↑
. . → → → ↑
. . . . . .
```